

Laboratorio 1 — Patrones de diseño de agentes con LLMs locales

Luis Seña Ávila, Universidad de Antioquia

Medellín - Antioquia

23 de septiembre de 2026

Resumen

En este laboratorio se estudiaron dos patrones de diseño de agentes basados en modelos de lenguaje (LLMs) que corren localmente con Ollama: el patrón de Reflexión (generar → criticar → corregir) y el patrón de Tool-calling (el LLM decide qué funciones de Python ejecutar para resolver una tarea). En el primero, un mismo modelo juega del rol de explicador y de revisor y se autocorriges antes de entregar la respuesta final. En el segundo, el agente recibe un conjunto de herramientas (listar datasets, ajustar curvas con `scipy`, graficar) y decide cuáles usar a partir de una instrucción en lenguaje natural. Se observó que ambos patrones funcionan correctamente con modelos pequeños locales como `llama3.2:1b` o `qwen3:1.7b`, pero que `llama3.2:1b` falla de formas instructivas: alucina argumentos que no existen en las herramientas, inventa números que no vienen de las funciones, y en el rol de revisor aprueba explicaciones incorrectas o rechaza explicaciones correctas por no respetar el formato pedido. La comparación entre modelos mostró que la calidad del agente depende críticamente del modelo subyacente, pero que el patrón de diseño se mantiene estable.

Palabras Claves: Agentes LLM, patrón de reflexión, tool-calling, Ollama.

1. Introducción

Un agente, en el contexto de los modelos de lenguaje, no es solamente un LLM que responde texto: es un sistema que recibe un objetivo, decide qué acciones tomar y las ejecuta, iterando hasta cumplirlo. La diferencia con un chatbot tradicional es que el agente *actúa*: llama funciones, se critica a sí mismo, incorpora los resultados y vuelve a decidir. Esta capacidad de actuar se construye combinando cuatro piezas: un modelo que razona, instrucciones de sistema que definen su rol, un conjunto de herramientas o roles adicionales, y un ciclo de control que conecta todo.

En este laboratorio se estudiaron dos formas distintas de armar ese ciclo, es decir, dos patrones de diseño. El primero, el patrón de Reflexión, hace que el mismo LLM juegue dos roles, un “explicador” que genera una respuesta y un “revisor” que la critica y se repite el ciclo hasta que el revisor apruebe la respuesta o se agoten los intentos. El segundo, el patrón de Tool-calling, le da al LLM un conjunto de herramientas de Python (listar datasets, ajustar un modelo matemático, graficar) y le pide que decida cuáles usar y en qué orden para resolver una instrucción dada en lenguaje natural.

En la Sección 2 se describe el marco conceptual de cada patrón y lo que se hizo en cada notebook. En la Sección 3 se comparan las ventajas y desventajas de ambos patrones, que es la pregunta central planteada para este laboratorio. En la Sección 4 se cierra con las conclusiones, resaltando los aprendizajes principales. Todo el trabajo se realizó con modelos corriendo 100 % localmente a través de [Ollama](#), sin ninguna API

key ni conexión a internet.

2. Desarrollo

2.1. ¿Qué es un agente LLM?

Un agente LLM es la combinación de cuatro componentes: (i) un modelo de lenguaje que razona, (ii) un *system prompt* que define su rol, (iii) un conjunto de herramientas o roles adicionales que puede usar, y (iv) un ciclo de control que conecta las salidas del modelo con las acciones del sistema. En un chatbot normal solo existen (i) y (ii). Añadir (iii) y (iv) es lo que convierte al modelo en un agente.

En este laboratorio el modelo subyacente corrió siempre en local con Ollama. Se usaron cuatro modelos según la sección: `llama3.2:1b` como modelo pequeño por defecto, `qwen3:1.7b` para comparar en Reflexión, y `qwen2.5:3b` en la sección final de Tools donde fue necesario uno más capaz.

2.2. Patrón de Reflexión: descripción

El patrón de Reflexión consiste en que el mismo LLM juegue dos roles distintos, con dos *system prompts* diferentes: un *explicador* que genera una primera respuesta y un *revisor* que la critica. Si el revisor rechaza, el explicador recibe los comentarios y produce una versión corregida. El ciclo se repite hasta que el revisor aprueba o se agotan los intentos.

La implementación usada (`reflection_agent.py`) define tres funciones: `explicar`, `revisar` y `corregir` y una función `run_reflection` que las encadena. El revisor emite su juicio en un formato fijo (`APROBADA: si/no` y `COMENTARIOS: ...`), y el código busca la cadena `.aprobada: si` para decidir si aprobó.

2.3. Patrón de Reflexión: qué se hizo en el laboratorio

Se corrió el notebook `reflexion_agent_lab.ipynb` con un concepto de física como caso de estudio: “*por qué el periodo de un péndulo simple no depende de la masa*”.

Línea base (sin reflexión). Con `llama3.2:1b` la primera explicación tuvo errores graves: escribió “pendú” en vez de “péndulo”, afirmó que “la masa del objeto sí influye en el periodo” (falso), e inventó que “el valor de g aumenta si aumentamos la masa”. Ninguno de esos errores fue detectado, simplemente porque no había revisor.

Ciclo paso a paso. Al revisar esa primera explicación, el revisor dijo `Aprobada: False` pero en el cuerpo de su respuesta escribió `APROBADA` sin el sufijo `: si`. El código, que busca la cadena exacta, lo interpretó como rechazo. Además, el comentario del revisor contenía él mismo un error conceptual: afirmó que “la relación entre masa y periodo es directa”. La corrección posterior, en lugar de mejorar la explicación, la empeoró.

Ciclo completo. Se corrieron hasta 3 rondas sin aprobación. En la ronda 1 el revisor propuso “aclarar la diferencia con un *mass point*” y en la ronda 2 pidió “expresar que la masa es una función polinómica de la masa”. Ambas sugerencias son incoherentes: el modelo pequeño, al no encontrar errores reales, inventa correcciones. Las explicaciones se volvieron progresivamente más confusas en lugar de más claras.

Varios conceptos. Con el concepto de caída libre el modelo inventó que “la masa de la pelota lanzada horizontalmente es $m + M$ ” y que por eso cae más rápido, salió aristotélico, y el revisor “aprobó” esa explicación. Con la diferencia entre velocidad y aceleración, después de 3 rondas el modelo nunca llegó a una definición correcta.

Comparación con un modelo más grande. Al cambiar solo el parámetro `model` por `qwen3:1.7b`, la primera ronda ya fue una explicación correcta, con la fórmula $T = 2\pi\sqrt{L/g}$ y la cancelación de la masa. El revisor, sin embargo, siguió reportando `APROBADA:` (con el formato incompleto) y el código lo siguió tratando como rechazo, por lo que se ejecutaron las 3 rondas. La calidad del texto, no obstante, fue muchísimo mejor: cada ronda mantuvo el nivel físico correcto.

2.4. Patrón de Tool-calling: descripción

El patrón de Tool-calling consiste en darle al LLM un conjunto de funciones de Python, y pedirle que decida

cuáles ejecutar para cumplir una instrucción. El ciclo es: el modelo razona sobre la petición, decide llamar a una herramienta con ciertos argumentos, el sistema ejecuta la función y le devuelve el resultado, y el modelo decide si llamar otra herramienta o dar la respuesta final.

En la implementación usada (`agent.py`), la lista de herramientas se pasa a `ollama.chat(..., tools=tools)` y el ciclo se implementa explícitamente: se itera hasta `max_turns`, ejecutando cada herramienta con los argumentos que el modelo proponga y añadiendo el resultado como un mensaje de rol `tool`. Las herramientas (`physics_tools.py`) permiten listar datasets, previsualizarlos, listar modelos de ajuste, ajustar una curva con `scipy.optimize.curve_fit` y generar una gráfica.

2.5. Patrón de Tool-calling: qué se hizo en el laboratorio

Se corrió el notebook `physics_agent_lab.ipynb` con tres datasets sintéticos (resorte, decaimiento RC, péndulo) y peticiones en lenguaje natural.

Ajuste del resorte con llama3.2:1b. El agente llamó correctamente a `fit_dataset({'dataset': 'resorte', 'model': 'lineal'})` y la herramienta devolvió $a = 25,70829 \pm 0,68034$ con $R^2 = 0,99267$. Sin embargo, en la respuesta final el agente *inventó* un valor distinto de k ($0,0424 \pm 0,0057$) y escribió una ecuación sin sentido. El error no está en la herramienta sino en la interpretación del resultado.

Ajuste del decaimiento con llama3.2:1b. Llamó correctamente a `fit_dataset` con el modelo `exponencial` y la herramienta devolvió $b = -0,51031 \pm 0,01838$ (que corresponde a $\tau \approx 1,96$ s). El agente reportó “ $\tau = \pm 0,01$ ”, otra vez un número inventado, no leído del JSON de la herramienta.

Caso sin la herramienta de graficar. Se le quitaron al agente las herramientas de graficación y se le pidió graficar el ajuste del péndulo. El agente intentó hacer el ajuste (lo cual sí podía) y al ver que no podía graficar respondió “no puedo generar gráficas”. Es un caso donde el modelo pequeño se comportó relativamente bien, aunque inventó una “varianza: 25,2” que no venía de ninguna herramienta.

El problema central: argumento alucinado en `plot_dataset_fit`. En la celda final se le dio al agente la lista completa de herramientas —incluyendo `plot_dataset_fit(dataset, model)`— y se le pidió generar la gráfica del ajuste de potencia para el péndulo. Con `llama3.2:1b` el agente llamó a la herramienta con un argumento adicional `type` que la función *no acepta*:

```
{.error": "plot_dataset_fit() got
an unexpected keyword argument
'type'}
```

Como consecuencia, la celda del notebook falló con

`KeyError: 'plot_path'` al intentar acceder a una clave que no existía en el JSON de respuesta. Se cambió el modelo por `qwen2.5:3b`, con el cual el agente llamó a la herramienta con los argumentos correctos, la herramienta devolvió la ruta del archivo, y la gráfica se mostró sin problemas. Este incidente ilustra una de las limitaciones más importantes del patrón de Tool-calling con modelos pequeños: un modelo de 1B parámetros no respeta con fiabilidad el esquema JSON de las herramientas y *inventa* parámetros que no existen. La solución es elegir un modelo con capacidad suficiente para tareas de tool-calling.

3. Ventajas y desventajas de cada patrón

3.1. Patrón de Reflexión

Ventajas:

- Mejora la calidad de la respuesta sin necesitar datos ni APIs externas: solo depende de la capacidad del propio modelo para criticarse.
- Es fácil de implementar: en el caso más simple, todo lo que se necesita son dos *system prompts* distintos (`EXPLICADOR_PROMPT` y `REVISOR_PROMPT`) y un bucle.
- Permite que el mismo modelo juegue roles distintos, aprovechando el conocimiento que ya tiene sin necesidad de entrenamiento adicional.

Desventajas:

- No agrega información nueva: el revisor solo puede criticar lo que el explicador ya escribió, sin acceso a fuentes externas. Si el modelo no sabe algo, ni el explicador ni el revisor lo van a saber.
- Si el revisor es débil, aprueba errores (o rechaza aciertos). Con `llama3.2:1b` se observó que el revisor aprobó explicaciones con física falsa y rechazó otras correctas solo por no respetar el formato.
- Puede entrar en bucles que no mejoran la respuesta. En el laboratorio, cada ronda con el modelo pequeño produjo textos más confusos, no mejores.

3.2. Patrón de Tool-calling

Ventajas:

- Conecta el lenguaje natural con acciones concretas: el agente puede leer archivos, ejecutar cálculos con `scipy` y generar gráficas, cosas que un LLM solo no puede hacer.
- El conjunto de herramientas define con precisión qué puede y qué no puede hacer el agente. Sin `plot_dataset_fit`, por ejemplo, el agente no puede graficar, sin importar cuán claro sea su “entendimiento” de la petición.

- Los resultados numéricos son reproducibles: vienen de Python y `scipy`, no del LLM, así que los valores de los ajustes son siempre los mismos aunque el LLM varíe entre corridas.

Desventajas:

- Depende de que las herramientas existan y estén bien definidas. El agente solo puede hacer lo que las funciones permiten.
- El modelo puede inventar números al interpretar el resultado de la herramienta. Se observó repetidamente con `llama3.2:1b`: el JSON de `fit_dataset` contenía $a = 25,71 \pm 0,68$, pero el agente reportaba $0,0424 \pm 0,0057$.
- El modelo puede alucinar argumentos que la herramienta no acepta, como ocurrió con el argumento `type` en `plot_dataset_fit`.

3.3. Comparación general

Los dos patrones son complementarios más que excluyentes. La Reflexión es útil cuando se quiere mejorar la calidad de un texto o respuesta sin acceso a información externa; el Tool-calling es útil cuando se necesita ejecutar acciones concretas y obtener resultados verificables. Un agente real podría combinar ambos: por ejemplo, un agente que ajusta una curva con herramientas y *después* reflexiona sobre si el modelo elegido tiene sentido físico.

La estabilidad de ambos patrones, sin embargo, depende críticamente del modelo subyacente. El mismo código que falla con `llama3.2:1b` funciona con `qwen2.5:3b` o `qwen3:1.7b`. La separación entre “patrón” y “modelo” es lo que permite cambiar uno sin tocar el otro.

4. Conclusiones

- Un agente no es solo un LLM. Es un LLM combinado con instrucciones, herramientas y un ciclo de control. El mismo `llama3.2:1b` se comportó como un “profesor que se autocritica” en el patrón de Reflexión y como un “asistente de laboratorio que ajusta datos” en el patrón de Tool-calling.
- El patrón de Reflexión no agrega información, solo la reformula. El revisor ve exactamente lo que el explicador escribió y no tiene acceso a ninguna fuente externa. Si el modelo base tiene un error conceptual, la reflexión no lo corrige —puede incluso empeorarlo, como se vio con `llama3.2:1b`.
- El patrón de Tool-calling sí agrega información nueva. Los números que devuelve `fit_dataset` vienen de `scipy`, no del LLM, y son por lo tanto reproducibles y verificables. Esa es la ventaja principal del patrón.
- Los modelos pequeños fallan de formas instructivas. Se observaron tres tipos de falla con `llama3.2:1b`: (i) alucinación de argumentos

(`type` en `plot_dataset_fit`), (ii) invención de números al interpretar los resultados de las herramientas ($k = 0,0424$ en vez de 25,71), y (iii) revisor que no respeta el formato pedido (APROBADA sin : `si`).

- Elegir el modelo adecuado es tan importante como diseñar bien el patrón. El mismo código, con `qwen2.5:3b` en lugar de `llama3.2:1b`, funcionó sin problemas en la generación de la gráfica. La separación entre la lógica del agente y el modelo subyacente es lo que permite este cambio sin tocar el resto del código.
- Los dos patrones son complementarios. La Reflexión mejora la forma; el Tool-calling agrega contenido verificable. Un agente real podría combinarlos, por ejemplo, ajustando un modelo con herramientas y luego reflexionando sobre si ese modelo tiene sentido físico para los datos.

5. Referencias

- [1] Ollama. <https://ollama.com>
- [2] SciPy: `scipy.optimize.curve_fit`. https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.curve_fit.html
- [3] N. Shinn et al. *Reflexion: Language Agents with Verbal Reinforcement Learning*. NeurIPS 2023.
- [4] S. Yao et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023.

Los notebooks completos con el desarrollo, los experimentos y los resultados pueden consultarse en los archivos `reflexion_agent_lab.ipynb` y `physics_agent_lab.ipynb` incluidos con este informe.

Declaración de uso de inteligencia artificial

Para la elaboración de este informe se utilizó la herramienta de inteligencia artificial DeepSeek (modelo de lenguaje) como apoyo en las siguientes tareas:

- Redacción y mejora de la claridad de las explicaciones en las secciones de desarrollo.
- Verificación de la coherencia entre los resultados de los notebooks y las conclusiones presentadas.
- Apoyo en la redacción de las secciones comparativas de ventajas y desventajas.

La ejecución de los notebooks, la observación de los resultados, la elección de los modelos y la interpretación de los fallos y aciertos fueron realizadas por mí. Las sugerencias proporcionadas por la IA fueron revisadas, validadas y adaptadas antes de su inclusión en el documento. El uso de esta herramienta se declara en cumplimiento de los principios de transparencia y honestidad académica.